

TURKCELL CODENIGHT 2026

FİNAL

NetOpsCell — Turkcell Şebeke Arıza Tahmini ve Saha Operasyon Platformu

Takım	3 Kişi
AI Araçları	Tamamen Serbest
Mimari	Mikroservis (zorunlu)
Backend	Zorunlu (serbest dil/framework)
Veritabanı	Servis başına ayrı DB (zorunlu)
Frontend	Web veya Mobil (en az 1 platform)
Deployment	Docker Compose (zorunlu)

İçindekiler

1. Proje Tanımı	3
1.1 Senaryo	3
1.2 Kullanıcı Roller	3
2. Mimari Gereksinimler	4
2.1 Zorunlu Mikroservisler	4
2.2 Mimari Kurallar	5
3. Identity Service — Fonksiyonel Gereksinimler	5
4. Incident Service — Fonksiyonel Gereksinimler	7
5. AI Service — Fonksiyonel Gereksinimler	9
6. Gamification Service — Fonksiyonel Gereksinimler	10
7. Süpervizör Dashboard	11
8. API Tasarım Beklentisi	12
9. Servisler Arası Event Akışı	12
10. Güvenlik Uyarısı	14
11. Kullanıcı Akışları ve Demo Senaryosu	15
12. Değerlendirme Kriterleri	16
13. Kurallar ve Diskalifiye Koşulları	17
14. Teslimat	17

1. Proje Tanımı

Türkiye'nin 7 bölgesinden finale kalan takımlar olarak, Turkcell şebeke altyapısındaki arızaları önceden tahmin eden ve saha ekiplerini yöneten, yapay zeka destekli, mikroservis mimarili bir şebeke operasyon platformu geliştireceksiniz. Bu final case'inde tek bir uygulama değil, birbiriyle konuşan bir sistem ekosistemi inşa edeceksiniz.

NetOpsCell dört bağımsız mikroservisten oluşur: kimlik ve yetki yönetimi (Identity), arıza yaşam döngüsü (Incident), yapay zeka tahmin motoru (AI) ve saha ekibi motivasyon sistemi (Gamification). Servisler bir API Gateway arkasında çalışır ve tüm sistem Docker Compose ile tek komutta ayağa kalkar.

1.1 Senaryo

Turkcell şebekesinde binlerce baz istasyonu ve altyapı bileşeni bulunur. Bir arıza ortaya çıktıktan sonra müdahale etmek, hizmet kesintisi ve müşteri mağduriyeti demektir. Arızayı önceden tahmin edip, en uygun saha ekibini doğru önceliklendirmeyeyle yönlendirmek, kesinti süresini düşürür ve şebeke kalitesini korur.

NetOpsCell bu süreci akıllı hale getirir: baz istasyonu telemetri verisi geldiğinde yapay zeka arıza olasılığını tahmin eder, arıza türünü sınıflandırır, öncelik atar ve en uygun saha ekibine yönlendirir. Saha ekipleri arızaları çözdükçe puan kazanır. Süpervizörler tüm şebeke sağlığını ve modelin doğruluğunu tek ekrandan izler.

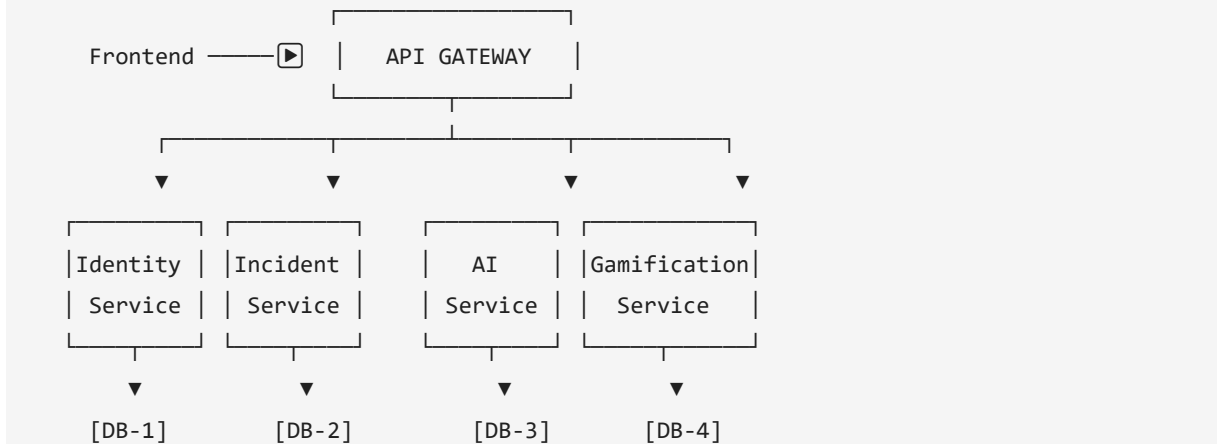
1.2 Kullanıcı Roller

Rol	Kim?	Ne Yapar?
Saha Teknisyeni	Sahada müdahale eden ekip	Atanan arızaları görür, müdahale eder, çözüm notu yazar, rozet kazanır
NOC Operatörü	Şebeke izleme çalışanı	Tahminleri görür, arızaları doğrular, saha ekibine yönlendirir
Süpervizör	Operasyon yöneticisi	Dashboard izler, model doğruluğunu ve SLA'yı takip eder, manuel atama yapar
Admin	Sistem yöneticisi	Ekip hesapları oluşturur, rol yönetir, audit log görür

2. Mimari Gereksinimler

2.1 Zorunlu Mikroservisler

Sistem en az 4 bağımsız mikroservis ve 1 API Gateway'den oluşmalıdır:



Servis	Sorumluluk
API Gateway	Tek giriş noktası. Routing, rate limiting, JWT doğrulama. Hazır gateway (Kong, Ocelot, Spring Cloud Gateway, Express Gateway) veya kendi yazdığınız reverse proxy kabul edilir.
Identity Service	Kayıt, giriş, token yönetimi, rol/yetki, audit log, hesap kilitleme.
Incident Service	Arıza yaşam döngüsü, saha ataması, SLA takibi, çözüm notu.
AI Service	Arıza tahmini, arıza türü sınıflandırma, akıllı saha ekibi ataması.
Gamification Service	Puan, rozet, seviye, liderlik. Event ile tetiklenir.

2.2 Mimari Kurallar

- Database-per-service: Her servis kendi veritabanına sahip olmalıdır. Bir servis başka servisin veritabanına doğrudan erişemez. DB tipi servise göre serbesttir (örn: Identity için PostgreSQL, Gamification için Redis+PostgreSQL).
- Servisler arası iletişim: REST çağrıları minimum kabul edilir. Message queue (RabbitMQ, Redis pub/sub, Kafka) kullanımı tercih edilir ve ekstra değerlendirilir.
- Bağımsızlık: Bir servis çöktüğünde diğer servisler çalışmaya devam etmelidir. Örneğin AI Service kapalıyken arıza oluşturulabilmeli (arıza vakası BELIRSIZ olarak işaretlenir, manuel atamaya düşer).
- Docker Compose: Tüm sistem (servisler + veritabanları + gateway) docker compose up komutuyla ayağa kalkmalıdır. Bu final seviyesinde zorunludur, bonus değildir.
- Her servisin kendi README'si olmalı: sorumluluk, endpoint listesi, environment değişkenleri.

3. Identity Service — Fonksiyonel Gereksinimler

3.1 Kayıt ve Giriş

- Müşteri/kullanıcı kaydı: Turkcell GSM numarası + OTP doğrulama (simülasyon: sabit kod 1234). Kayıt alanları: ad, soyad, GSM, e-posta (opsiyonel).
- Personel hesapları (analist/uzman/operatör/sorumlu ve süpervizör): Admin tarafından oluşturulur, e-posta + şifre ile giriş yapar. Oluştururken uzmanlık/bölge alanları atanır (birden fazla seçilebilir).
- Şifre politikası: minimum 8 karakter, en az 1 büyük harf, 1 rakam, 1 özel karakter. İhlalde hangi kuralın ihlal edildiğini belirten net hata mesajı dönmelidir.
- Şifreler bcrypt veya argon2 ile hash'lenmelidir. Düz metin veya MD5/SHA1 kabul edilmez.
- Hesap kilitleme: 5 başarısız girişte hesap 15 dakika kilitlenir. Kilitli hesaba girişte kalan süre bilgisi dönmelidir.

3.2 Token Yönetimi

- Access token: JWT, 15 dakika geçerlilik. Payload'da: user_id, rol, uzmanlık/bölge alanları.
- Refresh token: 7 gün geçerlilik, veritabanında saklanır.
- Token rotation: Refresh token kullanıldığında yeni refresh token üretilir, eskisi geçersiz kılınır. Geçersiz kılınmış bir refresh token tekrar kullanılmaya çalışılırsa o kullanıcının tüm oturumları sonlandırılır (token theft koruması).
- Logout: refresh token geçersiz kılınır.

3.3 Rol ve Yetki Matrisi

Aşağıdaki yetki matrisi endpoint seviyesinde uygulanmalıdır. Yetkisiz erişim denemesi 403 dönmeli ve audit log'a yazılmalıdır:

İşlem	Müşteri	Personel	Süpervizör	Admin
Arıza oluşturma	✓	—	—	—
Kendi kayıtlarını görme	✓	✓ (atanan)	✓ (tümü)	✓ (tümü)
Durum değiştirme	—	✓	✓	—
Manuel atama	—	—	✓	—
Kategori/tür değiştirme (AI override)	—	✓	✓	—
Dashboard görüntüleme	—	—	✓	✓
Personel hesabı oluşturma	—	—	—	✓
Audit log görüntüleme	—	—	—	✓

3.4 Audit Log

Aşağıdaki işlemler audit log'a kaydedilmelidir:

- Başarılı ve başarısız giriş denemeleri

- Hesap kilitlenmesi
- Rol değişiklikleri
- Yetkisiz erişim denemeleri (403)
- Arıza silme ve kritik durum değişiklikleri

Her log kaydında bulunması gereken alanlar: kim (user_id), ne (işlem tipi), ne zaman (timestamp), nereden (IP), sonuç (başarılı/başarısız), detay (ilgili kaynak id'si).

4. Incident Service — Fonksiyonel Gereksinimler

4.1 Arıza Oluşturma

- Arıza/telemetri girdisi: baz istasyonu kodu, konum, metrikler (sinyal gücü, paket kaybı, sıcaklık, güç durumu). Gerçek istasyon entegrasyonu beklenmez; telemetri simülatörü veya form girişi ile üretilir.
- Her telemetri kaydı AI Service'e gönderilir: arıza olasılığı, arıza türü ve öncelik atanır. Yüksek olasılıklı olanlar arıza vakasına dönüşür.
- AI Service erişilemez durumdaysa vaka yine oluşturulmalı (tür: BELIRSIZ, öncelik: ORTA) ve manuel atama kuyruğuna düşmelidir.
- Arıza numarası: benzersiz, okunabilir (örn: INC-2026-000123).

4.2 Arıza Vakası Yaşam Döngüsü

Yüksek olasılıklı arızalar bir vakaya dönüşür. Durumlar ve geçiş kuralları aşağıdaki state machine'e uymalıdır. Kural dışı geçiş 422 dönmelidir:

Mevcut Durum	Hedef Durum	Kim Yapabilir	Koşul
YENI	ATANDI	Sistem (AI) / Süpervizör	Saha ekibi belirlendi
ATANDI	YOLDA	Saha Teknisyeni	Ekip sahaya hareket etti
YOLDA	MUDAHALE_EDILIYOR	Saha Teknisyeni	Ekip sahaya ulaştı
MUDAHALE_EDILIYOR	PARCA_BEKLENIYOR	Saha Teknisyeni	Yedek parça gerekiyor
PARCA_BEKLENIYOR	MUDAHALE_EDILIYOR	Sistem	Parça tedarik edildi
MUDAHALE_EDILIYOR	COZULDU	Saha Teknisyeni	Çözüm notu zorunlu
COZULDU	KAPANDI	NOC / Sistem	Doğrulama veya 24 saat

4.3 Arıza Türleri ve Öncelik

- Türler/kategoriler: DONANIM, GUC_KESINTISI, BAGLANTI, YAZILIM, ISINMA, BELIRSIZ.
- Arıza türü AI tarafından atanır; NOC operatörü veya süpervizör değiştirebilir.
- Operatör türü değiştirdiğinde bu değişiklik AI Service'e bildirilmelidir (doğruluk metriği için).
- Öncelik seviyeleri: DUSUK, ORTA, YUKSEK, KRITIK. AI etkilenen kullanıcı sayısı ve arıza olasılığına göre atar: büyük kapsama alanı + yüksek olasılık → KRITIK. Süpervizör manuel değiştirebilir.

4.4 SLA Kuralları

Öncelik	SLA Süresi	Aşım Durumunda
KRITIK	1 saat	Vaka kırmızı işaretlenir, süpervizör panelinde en üstte görünür

YUKSEK	4 saat	Vaka turuncu işaretlenir
ORTA	12 saat	Görsel uyarı
DUSUK	48 saat	Görsel uyarı

SLA süresi vaka oluşturma anından itibaren sayılır. COZULDU durumuna geçince durur. Kalan SLA hem saha ekibi hem süpervizör ekranında görünür olmalıdır.

4.5 Saha İletişimi

- Saha teknisyeni ve NOC operatörü vaka üzerinden mesajlaşabilmelidir (thread yapısı).
- Saha ekibi ek bilgi/parça talep ettiğinde durum PARCA_BEKLENIYOR'a çekilebilir; tedarik olunca MUDAHALE_EDILIYOR'a döner.
- Her mesajda gönderen, zaman damgası, içerik bulunur.

4.6 Çözüm Değerlendirmesi

- Vaka KAPANDI durumuna geçtikten sonra NOC operatörü çözümü 1-5 yıldız değerlendirir (kalıcı çözüm mü, geçici mi).
- Değerlendirme tek seferliktir. Verildiğinde Gamification Service'e event gönderilir.

5. AI Service — Fonksiyonel Gereksinimler

Bu servis final case'inin kalbidir. Üç görevi vardır ve her üçü de zorunludur.

5.1 Görev 1: Arıza Tahmini

- Girdi: baz istasyonu telemetri verisi (sinyal, paket kaybı, sıcaklık, güç, geçmiş arıza). Çıktı: arıza olasılığı (0.0-1.0) + öneri (IZLE < 0.4, VAKA_AC 0.4-0.85, ACIL 0.85+).
- Olasılık 0.40 altı sadece izlenir, 0.85 üzeri acil vaka açılır. Aradakiler operatör onayına düşer.
- Yaklaşım serbesttir: (a) kendi eğittiğiniz klasik ML modeli (scikit-learn vb.), (b) kural tabanlı + ML hibrit, (c) LLM API entegrasyonu. Hepsi kabul edilir.
- Kendi veri setinizle model eğiten takımlar ekstra değerlendirilir. Bu durumda eğitim verinizi repository'de paylaşmanız ve README'de eğitim sürecini anlatmanız gerekir.
- Eğitim/test verisini kendiniz oluşturacaksınız: gerçekçi Türkçe normal ve arızalı telemetri örnekleri (örn: 'sinyal stabil, sıcaklık normal', 'sıcaklık hızla yükseliyor + paket kaybı artıyor'). Minimum 100 örnek önerilir; AI araçlarıyla üretebilirsiniz.

5.2 Görev 2: Arıza Türü Sınıflandırma

- Girdi: telemetri paterni. Çıktı: DONANIM, GUC_KESINTISI, BAGLANTI, YAZILIM, ISINMA.
- Tür, saha ekibinin hangi parça/ekipmanla gideceğini belirlemesine yardımcı olur; ekranında gösterilmeli.

5.3 Görev 3: Akıllı Saha Ekibi Ataması

Kategorisi/skoru belirlenen arıza vakası, uygun kişiye otomatik atanmalıdır. Atama bir skortlama algoritmasına dayanmalıdır. Örnek formül (kendi formülünüzü tasarlayabilirsiniz):

$$\text{skor} = (\text{uzmanlik_eslesme} \times 0.4) + (\text{mesafe_yakınlık} \times 0.3) + (\text{bosluk_orani} \times 0.3)$$

- uzmanlik_eslesme: Ekibin uzmanlığı (örn: donanım) arıza türüyle eşleşiyorsa 1, değilse 0.
- mesafe_yakınlık: Ekibin arıza konumuna yakınlığı (yakın ekip öncelikli). Haversine ile hesaplanabilir.
- bosluk_orani: $1 - (\text{aktif arıza} / \text{maksimum kapasite})$. Kapasite ekip başına 5 aktif arıza.
- En yüksek skorlu ekibe atama yapılır. Kapasite yoksa vaka kuyruğa alınır.
- Süpervizör her zaman manuel atama yapabilir.

5.4 Doğruluk Takibi

- Personel veya süpervizör AI'ın atadığı kategori/türü değiştirirse bu 'yanlış sınıflandırma' olarak kaydedilmelidir.
- Süpervizör dashboard'unda AI doğruluk oranı gösterilmelidir: $\text{doğru} / \text{toplam} \times 100$.
- Kategori bazlı doğruluk kırılımı (hangi türde ne kadar isabetli) ekstra değerlendirilir.

6. Gamification Service — Fonksiyonel Gereksinimler

Personelin motivasyonunu artıran puan, rozet, seviye ve liderlik sistemi. Bu servis Incident Service'ten gelen event'lerle çalışır — doğrudan çağrı değil, olay tabanlı mimari beklenir.

6.1 Puan Tablosu

Olay	Puan	Koşul
Arıza çözüldü (COZULDU)	+10	Her çözüm
Hızlı müdahale bonusu	+5	SLA'nın yarısından kısa
Kalıcı çözüm (5 yıldız)	+10	NOC değerlendirmesi
KRITİK arıza SLA içinde çözüldü	+15	SLA içinde
SLA aşımı	-5	Her aşım
Tekrar eden arıza (24 saatte aynı istasyon)	-3	Geçici çözüm

6.2 Rozetler

Rozet	Kazanılma Koşulu
İlk Müdahale	İlk arızayı çözme
Hız Ustası	SLA'nın yarısında 10 müdahale
Kalıcı Çözüm	20 arızada tekrar olmadan
Maratoncu	Bir günde 15 arıza çözümü
Kriz Yöneticisi	10 KRITİK arızayı SLA içinde çözme
Uzman	Tek türde 50 arıza çözümü

6.3 Seviye Sistemi

Seviye	Puan Aralığı	Görsel
Bronz	0 – 499	Bronz rozet/çerçeve
Gümüş	500 – 1.499	Gümüş rozet/çerçeve
Altın	1.500 – 2.999	Altın rozet/çerçeve
Platin	3.000+	Platin rozet/çerçeve

6.4 Liderlik Tablosu ve Profil

- Günlük ve haftalık liderlik tablosu: ilk 10 kişi, puan sıralı.
- Liderlik tablosu gerçek zamanlı veya sayfa yenilemede güncel olmalıdır.
- Profil ekranı: toplam puan, seviye, kazanılan rozetler, günlük/haftalık sıralama, çözülen vaka sayısı, ortalama puan.
- Rozet kazanıldığı anda kişiye görsel bildirim gösterilmelidir (toast/modal).

7. Süpervizör Dashboard

Süpervizör rolündeki kullanıcılar için operasyonel görünürlük ekranı. Aşağıdaki bileşenler zorunludur:

- Arıza dağılımı: tür bazlı pasta/bar grafik
- Öncelik dağılımı ve trend
- SLA uyum oranı + SLA aşmış aktif arızalar
- AI doğruluk metriği: arıza tahmini isabet oranı (false alarm oranı dahil)
- Saha ekibi performansı: çözülen arıza, ortalama müdahale süresi, tekrar oranı
- Bekleyen atama kuyruğu: BELIRSIZ veya kapasite bekleyen, manuel atama

8. API Tasarım Beklentisi

Bu dokümanda tüm endpoint listesi verilmemektedir. API'nizi RESTful prensiplere uygun olarak kendiniz tasarlayacaksınız — tasarım kaliteniz değerlendirme konusudur. Aşağıda beklentiye netleştiren örnekler verilmiştir.

8.1 Gateway Routing Örneği

```
/api/v1/auth/**      → Identity Service
/api/v1/incidents/** → Incident Service
/api/v1/ai/**        → AI Service
/api/v1/game/**      → Gamification Service
```

8.2 Örnek Endpointler

Method	Endpoint	Açıklama
POST	/api/v1/telemetry	Telemetri gönder (AI arıza tahmini tetiklenir)
POST	/api/v1/ai/predict	Arıza tahmini (Incident Service çağırır)
GET	/api/v1/game/leaderboard?period=daily	Günlük liderlik tablosu
PATCH	/api/v1/incidents/:id/status	Arıza durumu güncelle (saha)

Geri kalan tüm endpoint'leri kendiniz tasarlayın. Standart response formatı kullanın ({ success, data, error }) ve API'nizi Swagger/OpenAPI ile dokümente edin.

9. Servisler Arası Event Akışı

Servisler arası iletişimde olay (event) tabanlı tasarım beklenir. Aşağıda bir uçtan uca örnek verilmiştir; diğer tüm event'leri kendiniz tasarlayıp dokümente edeceksiniz. Event tasarımınız değerlendirme konusudur.

9.1 Örnek: 'incident.resolved' Akışı

1. Saha teknisyeni arızayı COZULDU durumuna çeker (Incident Service) → 2. Incident Service 'incident.resolved' event'i yayınlar → 3. Gamification Service dinler, müdahale süresini hesaplar → 4. Puan ekler (+10, hızlıysa +5) → 5. Rozet koşullarını kontrol eder, 'badge.earned' üretir → 6. Frontend'e bildirim yansır

9.2 Örnek Event Payload

```
{
  "event_type": "incident.resolved",
  "timestamp": "2026-07-18T09:41:03Z",
  "payload": {
    "incident_id": "INC-2026-000123",
    "team_id": "a7f3...",
    "fault_type": "ISINMA",
    "priority": "KRITIK",
    "created_at": "2026-07-18T08:50:11Z",
    "resolved_at": "2026-07-18T09:41:03Z"
  }
}
```

```
}  
}
```

Tasarlamanız gereken diğer olaylara örnekler: arıza tahmin edildi, tür değiştirildi (AI doğruluk takibi), parça tedarik edildi, SLA aşıldı. Event'lerinizi README veya ayrı bir EVENTS.md dosyasında dokümente edin.

10. Güvenlik Uyarısı

JÜRİ CANLI GÜVENLİK TESTİ YAPACAKTIR

Demo sırasında jüri üyeleri sisteminize kasıtlı saldırı senaryoları deneyecektir. Hazırlıklı olun.

Denenecek senaryolara örnekler (tam liste değildir):

- SQL injection: form alanlarına ' OR 1=1 -- benzeri girdiler
- Yetkisiz endpoint erişimi: müşteri token'ıyla süpervizör endpoint'i çağırma
- Başka kullanıcının verisine erişim: kayıt ID değiştirerek başkasının verisini görme (IDOR)
- Token manipölasyonu: süresi dolmuş veya değiştirilmiş JWT ile istek
- Geçersiz kılınmış refresh token'ın yeniden kullanımı
- XSS: metin alanına script etiketi enjeksiyonu
- Brute-force: ardışık hızlı giriş denemeleri (rate limit testi)

11. Kullanıcı Akışları ve Demo Senaryosu

11.1 Saha Teknisyeni Akışı

1. Uygulamaya giriş → 2. Atanan arızayı gör (konum + AI türü) → 3. Sahaya hareket et (durum YOLDA) → 4. Müdahale et → 5. Çözüm notuyla kapat → 6. Puan ve rozet kazan

11.2 NOC Operatörü Akışı

1. Panele giriş → 2. Arıza tahminlerini gör → 3. Yüksek olasılıklı tahmini doğrula → 4. Vaka aç ve saha ekibine ata → 5. Müdahale sürecini takip et → 6. Çözümü değerlendir

11.3 Zorunlu Demo Senaryosu

Sunumunuzda aşağıdaki uçtan uca akışı canlı göstermeniz zorunludur:

1. docker compose up ile tüm sistemi ayağa kaldır → 2. Kritik telemetri verisi oluştur (sinyal çöktü, sıcaklık yüksek) → 3. AI'nın arıza olasılığı + tür + öncelik atamasını göster → 4. Arızanın doğru uzmanlıktaki (ve yakın) saha ekibine atandığını göster → 5. Saha teknisyeni olarak arızayı çöz → 6. Puanın liderlik tablosuna yansıdığını göster → 7. Bir servisi kapat (docker stop) ve sistemin geri kalanının çalıştığını kanıtla → 8. Jürinin güvenlik testlerine hazır ol

Servis kapatma adımı final jürisinin en çok önem verdiği andır: mikroservis mimarinizin gerçekten bağımsız çalıştığını burada kanıtlarsınız.

12. Değerlendirme Kriterleri

Toplam puan 100 üzerinden hesaplanır. Final değerlendirmesi 6 kategoride yapılır:

Kategori	Ağırlık	Değerlendirme Detayı
Mimari ve Kod Kalitesi	25 puan	Mikroservis bağımsızlığı, database-per-service uyumu, event tasarımı, gateway kurgusu, clean code, SOLID, anlamlı commit geçmişi
Fonksiyonellik	25 puan	Dört servisin zorunlu özelliklerinin çalışması, AI görevleri, state machine, gamification kuralları, edge case yönetimi
Güvenlik	15 puan	Jüri güvenlik testlerinden geçme, token rotation, rol yetki matrisi uyumu, audit log, rate limiting
UI/UX Kalitesi	10 puan	Görsel tutarlılık, responsive tasarım, loading/error/empty state, kullanıcı deneyimi akışı
Test ve Dokümantasyon	10 puan	Unit/integration test, Swagger/OpenAPI, README'ler, EVENTS.md, AI yaklaşım dokümanı
Sunum ve Canlı Demo	15 puan	Zorunlu demo senaryosunun eksiksiz gösterimi (servis kapatma dahil), mimari anlatımı, teknik kararların savunulması

12.1 Bonus Puanlama (+20 puan)

Aşağıdaki her özellik ek puan getirir. Toplam bonus 20 puanı aşamaz.

- Kendi eğittiğiniz ML modeli (eğitim verisi + süreç dokümante edilmiş) (+8)
- Message queue ile event iletimi (RabbitMQ, Kafka, Redis pub/sub) (+5)
- Kategori bazlı AI doğruluk kırılımı (süpervizör panelinde) (+3)
- Gerçek zamanlı bildirimler (WebSocket/SSE) (+2)
- CI/CD pipeline (GitHub Actions vb.) (+2)

13. Kurallar ve Diskalifiye Koşulları

13.1 Genel Kurallar

- Takım: 3 kişi, değiştirilemez
- Tüm takımlar aynı case'i çözer
- AI araçları (Copilot, ChatGPT, Claude, Cursor vb.) tamamen serbest
- Boş proje scaffold/boilerplate ile başlamak serbest
- Önceden yazılmış iş mantığı kodu ile başlamak yasak

13.2 Diskalifiye Koşulları

- Monolith mimari (tek servis, tek uygulama) değerlendirme dışıdır
- docker compose up ile sistem ayağa kalkmıyorsa değerlendirme dışıdır
- AI servisi mock/hardcoded ise (her girdiye sabit çıktı) değerlendirme dışıdır
- Servisler tek/ortak veritabanı paylaşıyorsa değerlendirme dışıdır
- Çalışmayan (build edilemeyen) projeler değerlendirme dışıdır

14. Teslimat

14.1 Kod

- GitHub/GitLab repository (public veya davetli erişim). Monorepo veya servis başına ayrı repo — ikisi de kabul.
- Ana branch'te çalışır durumda kod, anlamlı commit geçmişi
- Kök dizinde docker-compose.yml: tüm servisler + veritabanları + gateway
- .env.example dosyaları (servis başına)

14.2 Dokümantasyon

- Ana README: sistem genel bakış, mimari diyagram, kurulum (docker compose up + seed), demo kullanıcı bilgileri
- Servis başına README: sorumluluk, endpoint listesi, environment değişkenleri
- EVENTS.md: tasarladığınız tüm event'ler ve payload'ları
- AI yaklaşım dokümanı: hangi yöntemi seçtiniz, neden, nasıl çalışıyor. Model eğittiyseniz eğitim verisi ve süreci
- Swagger/OpenAPI dokümantasyonu (en az Incident Service ve AI servisleri için)

14.3 Sunum (Maks. 15 Dakika)

- Canlı demo (6-7 dk): Zorunlu demo senaryosunun tamamı (bölüm 11.3)
- Mimari anlatımı (4 dk): servis sorumlulukları, event akışı, AI yaklaşımı, güvenlik önlemleri
- Zorluklar ve çözümler (2 dk)
- Soru-cevap ve güvenlik testi (jüri yönetiminde)

Sorularınız için mentorlara danışabilirsiniz. Mentorlar teknik yönlendirme yapabilir ancak kod yazamazlar.

Finalde başarılar. 🚧 Şebeke ayakta, saha hazır.